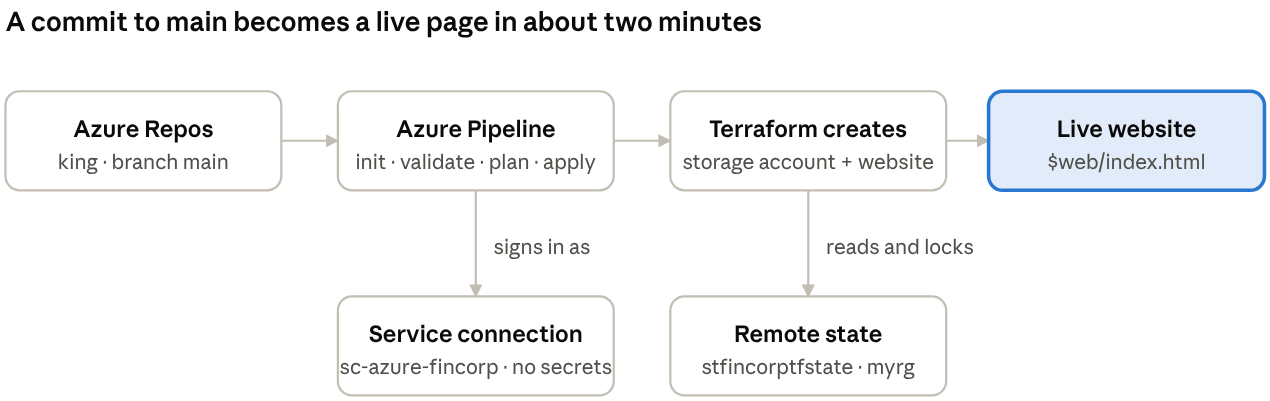
FinCorp Terraform Pipeline — End-to-End Guide

Nov 3, 2026 · @naveen

Overview

This guide builds a complete infrastructure-as-code pipeline on Azure: Terraform files in an Azure Repos Git repository, an Azure Pipeline that runs `init → validate → plan → apply` on every push to `main`, and a static website on Azure Blob Storage as the deployed result. Nothing is created by hand in the portal after setup.



deployment flow · 4 steps, 2 supporting services

The pipeline authenticates to Azure through a workload-identity service connection (no stored secrets) and keeps Terraform state in a dedicated storage account so every run sees the same picture of what already exists.

Prerequisites

Two accounts and one local tool are enough; everything else is created in the steps below.

Item	Value used in this guide	Where it lives
Azure subscription	Azure subscription 1 (`fc6f4b31-…-b6f8b776b00f`)	portal.azure.com

Item	Value used in this guide	Where it lives
Azure DevOps organization	`naveenkolli7`	dev.azure.com
Azure DevOps project / repo	`king` / `king`	Repos
State storage account	`stfincorptfstate`, container `tfstate`, resource group `myrg`	Azure
Service connection	`sc-azure-fincorp`	Project Settings
Website storage account	`naveenweb2026` in `rg-fincorp-dev`, Central US	created by Terraform
Git client	Git Bash (Windows) or Terminal (Mac)	local machine

Naming rule to remember: a storage account name is 3–24 characters, lowercase letters and digits only, and unique across all of Azure.

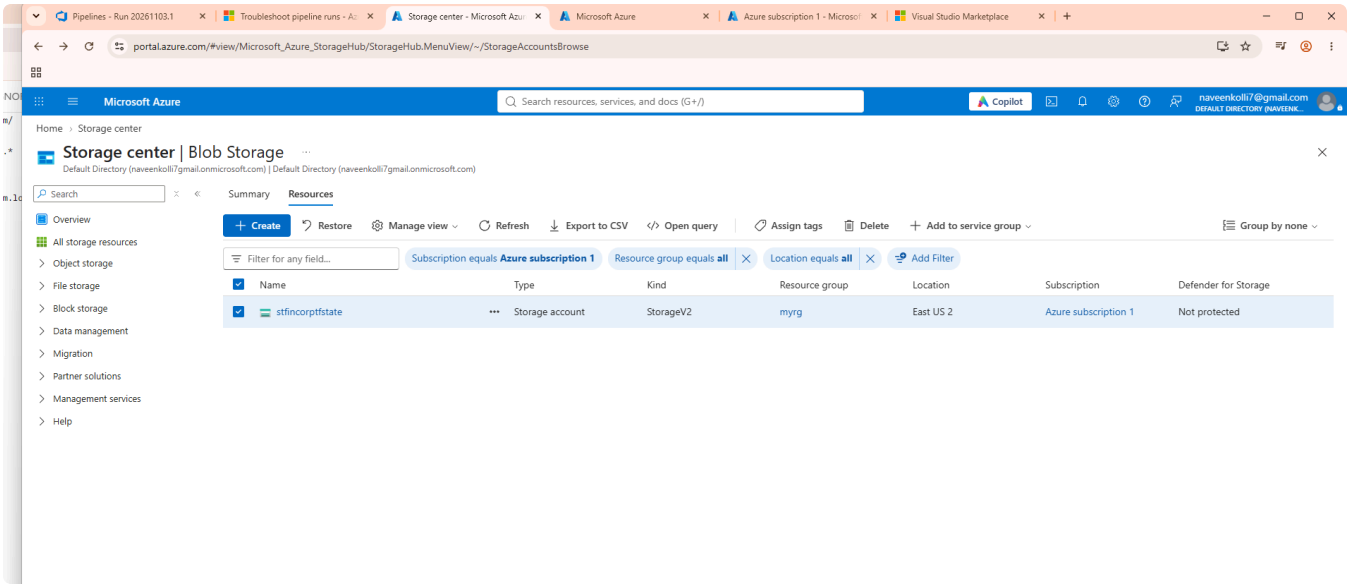
Step 1 – Azure: create the state storage account

Terraform must keep its state file somewhere every pipeline run can reach, so one storage account is created by hand before any code runs. This is the only resource made in the portal.

1. Sign in at portal.azure.com with the account that owns the subscription.
2. In the top search bar type **Storage accounts** → open it → click **+ Create**.
3. **Basics** tab:
   - Subscription: `Azure subscription 1`
   - Resource group: click **Create new** → type `myrg` → OK
   - Storage account name: `stfincorptfstate` (if the portal shows a red “already taken”, add digits, e.g. `stfincorptfstate01`, and use that name everywhere below)
   - Region: `(US) East US 2`
   - Primary service: Azure Blob Storage
   - Performance: Standard
   - Redundancy: Locally-redundant storage (LRS)
4. Click **Review + create** (skip the other tabs) → wait for “Validation passed” → **Create**. Deployment takes about 30 seconds.

5. Click **Go to resource**.
6. Left menu → **Data storage** → **Containers** → **+ Container**.
7. Name: `tfstate`. Anonymous access level: Private. Click **Create**.
8. Still on the storage account, open **Settings → Configuration** and confirm **Allow storage account key access** is *Enabled* (default). Terraform uses keys to read state.

Write down the three names (resource group, storage account, container) exactly as created. They are typed into `backend.tf` and the pipeline YAML in later steps, and the most common failure in this whole build was a mismatch here.



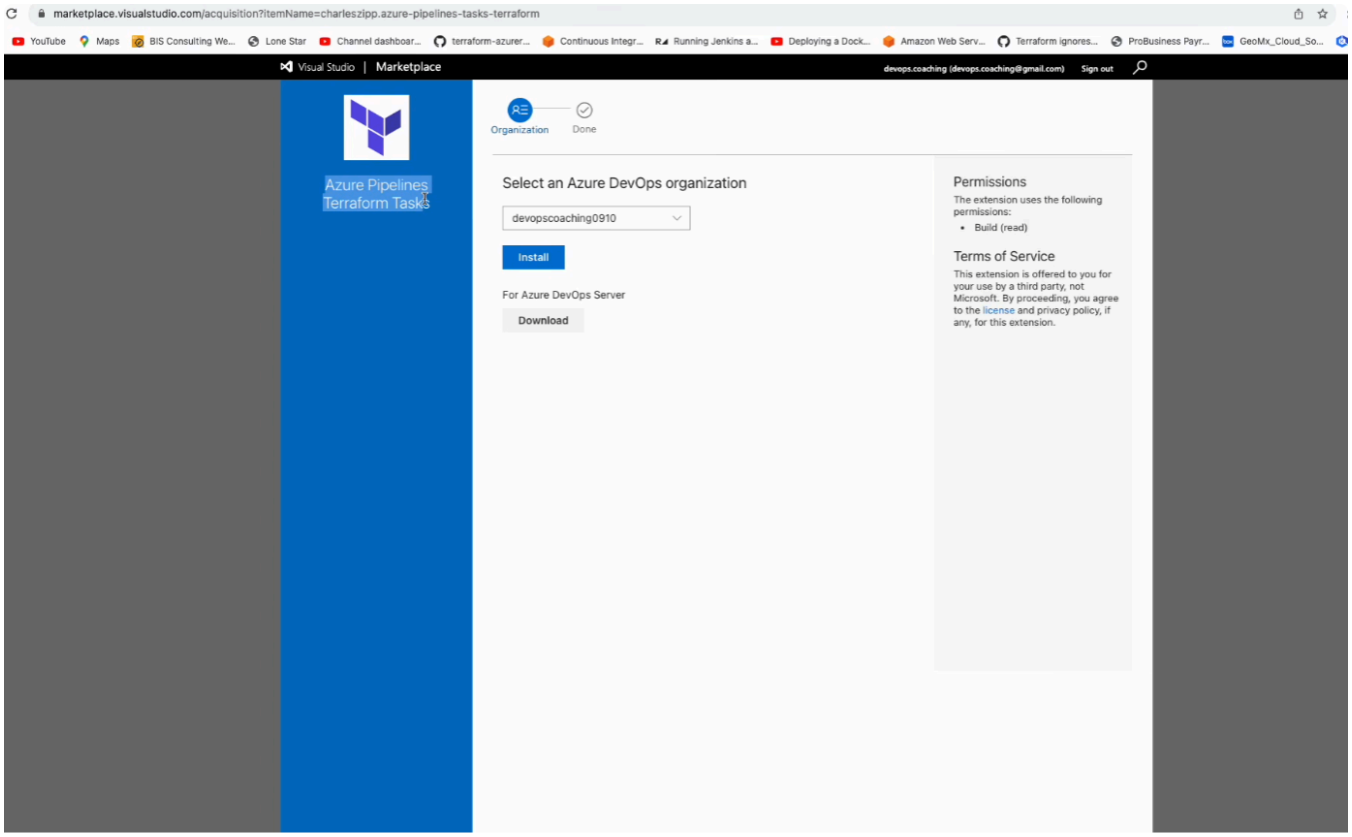
Step 2 – Azure DevOps: project, repo and Terraform extension

Azure DevOps holds the code and runs the pipeline; one marketplace extension gives the pipeline its Terraform tasks.

1. Sign in at dev.azure.com with the same Microsoft account. If you have no organization yet, click **Create new organization**, accept the defaults, and name it `naveenkolli7`.
2. Click **+ New project**. Project name: `king`. Visibility: **Private**. Leave Advanced as is → **Create**.
3. Left menu → **Repos** → **Files**. The repo `king` exists and is empty; this is where the Terraform files go in Step 3.
4. Install the Terraform tasks: open Terraform by Microsoft DevLabs in a new tab → click **Get it free**.
5. On the install page pick organization `naveenkolli7` in the dropdown → **Install** → **Proceed to organization**.

6. Confirm: dev.azure.com → **Organization settings** (bottom-left) → **Extensions** → **Installed** tab lists "Terraform". The pipeline tasks `TerraformInstaller@1` and `TerraformTaskV4@4` are now available to every project in the organization.

Use the Microsoft DevLabs extension, not the older "Azure Pipelines Terraform Tasks" by Charles Zipp / Jason Johnson: that one has been unpublished and its task names (`TerraformCLI@1`) do not match this guide.



ɔrm + Azure DevOps: Automate Azure Cloud Infra Setup

Step 3 – Terraform code: seven files at the repo root

Each file has one job. Keep them all in the root of the `king` repo, next to each other.

File	Job
`azure.tf`	Which provider and version Terraform downloads
`backend.tf`	Where the state file lives (the account from Step 1)
`variables.tf`	The inputs the code accepts
`terraform.tfvars`	The actual values for those inputs

File	Job
`webapp.tf`	The resources: resource group, storage account, static website, home page
`index.html`	The web page that gets uploaded
`.gitignore`	Keeps state and `.terraform/` out of Git

azure.tf

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

backend.tf — the three names must match Step 1 exactly.

```
terraform {
  backend "azurerm" {
    resource_group_name  = "myrg"
    storage_account_name = "stfincorptfstate"
    container_name       = "tfstate"
    key                  = "webapp.tfstate"
  }
}
```

variables.tf

```
variable "resource_group_name" {
  type        = string
  description = "Resource group for the website"
}

variable "location" {
  type    = string
  default = "centralus"
}

variable "app_name" {
  type        = string
  description = "Storage account name: 3-24 chars, lowercase letters and
digits only, globally unique"
}

variable "environment" {
  type    = string
  default = "dev"
}
```

terraform.tfvars — change `app_name` to your own unique value.

```
resource_group_name = "rg-fincorp-dev"
location            = "centralus"
app_name            = "naveenweb2026"
environment         = "dev"
```

webapp.tf

```
# Create a resource group
resource "azurerm_resource_group" "dev-rg" {
  name     = var.resource_group_name
  location = var.location
}

# Storage account hosting the site
resource "azurerm_storage_account" "web" {
  name                     = var.app_name
  resource_group_name      = azurerm_resource_group.dev-rg.name
  location                 = azurerm_resource_group.dev-rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
```

```
  tags = {
    environment = var.environment
  }
}

# Turn on static website hosting
resource "azurerm_storage_account_static_website" "web" {
  storage_account_id = azurerm_storage_account.web.id
  index_document     = "index.html"
}

# Upload the home page; content_md5 makes Terraform re-upload when the file
changes
resource "azurerm_storage_blob" "index" {
  name                   = "index.html"
  storage_account_name   = azurerm_storage_account.web.name
  storage_container_name = "$web"
  type                   = "Block"
  content_type           = "text/html"
  source                 = "${path.module}/index.html"
  content_md5            = filemd5("${path.module}/index.html")
  depends_on             = [azurerm_storage_account_static_website.web]
}

output "website_url" {
  value = azurerm_storage_account.web.primary_web_endpoint
}
```

.gitignore

```
.terraform/
*.tfstate
*.tfstate.*
crash.log
*.tfplan
.terraform.lock.hcl
```

index.html — any HTML page. The one deployed in this build is the FinCorp landing page shown in Step 6.

Putting the files in the repo — option A, in the browser (no Git needed)

1. Azure DevOps → **Repos** → **Files**. On an empty repo click **Initialize** with the default README (or skip if files already exist).
2. Click the ⋮ next to the repo name → **+ New** → **File**. Type the file name exactly, e.g. `azure.tf` → **Create**.
3. Paste the file contents into the editor → **Commit** (top right) → keep the default message → **Commit**.
4. Repeat for `backend.tf`, `variables.tf`, `terraform.tfvars`, `webapp.tf`, `index.html`, `.gitignore`.
5. Open each file once and check the first and last lines match the code above; a missed closing `}` is the most common paste mistake.

Option B, from Git Bash on your machine

1. Azure DevOps → **Repos** → **Clone** (top right) → copy the HTTPS URL → click **Generate Git Credentials** and copy the password shown.
2. Open Git Bash and run, one line at a time:

```
git config --global user.name "Naveen Kolli"
git config --global user.email "naveenkolli7@gmail.com"
git clone https://naveenkolli7@dev.azure.com/naveenkolli7/king/_git/king
cd king
```

3. When the clone asks for a password, paste the generated credential.
4. Copy the seven files into the `king` folder (File Explorer / Finder is fine). `ls -a` should list all seven.
5. Push, one line at a time:

```
git add .
git commit -m "Add Terraform static website"
git branch -M main
git push -u origin main
```

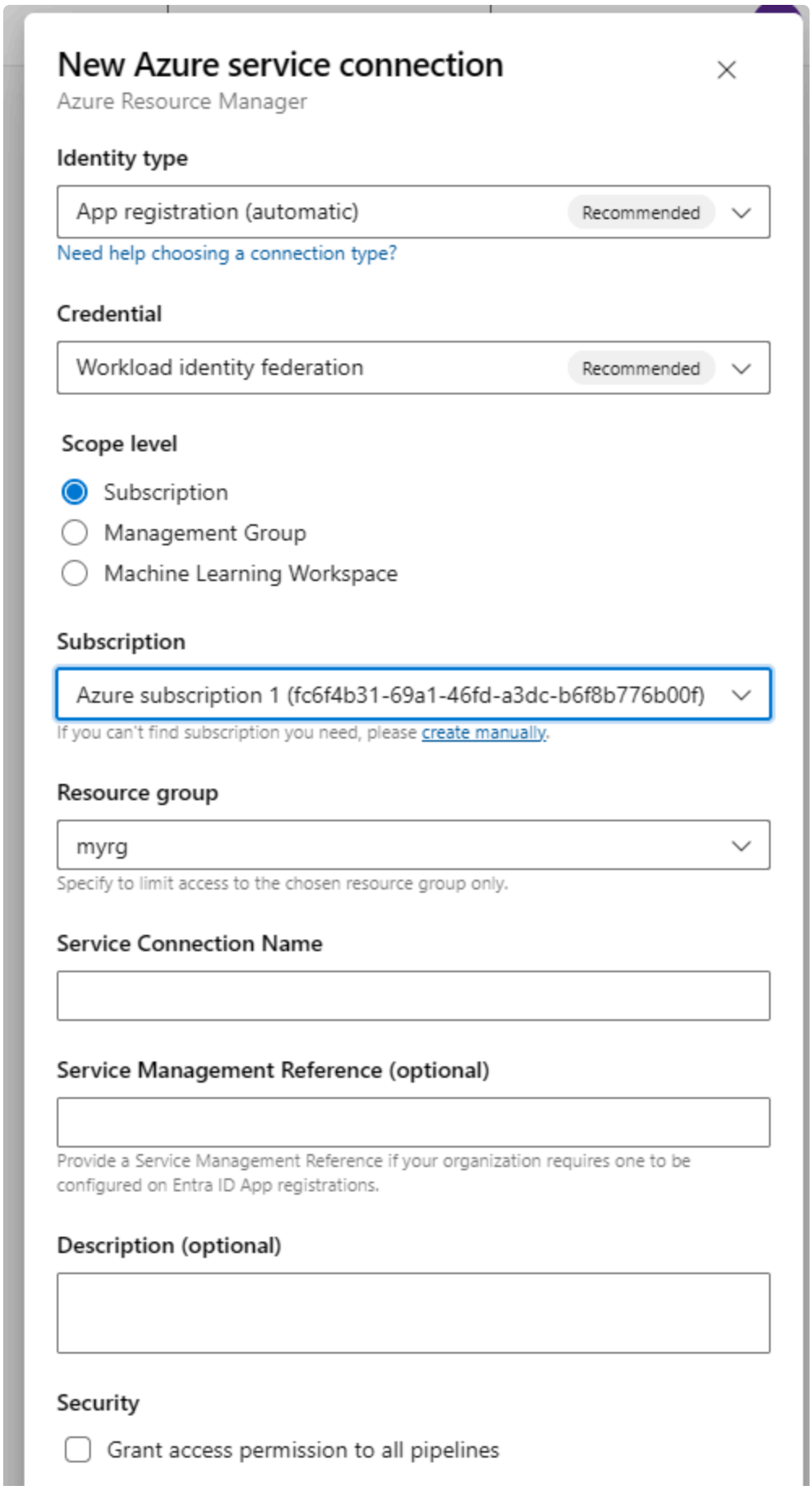
6. Refresh **Repos → Files** in the browser; the seven files appear on branch `main`.

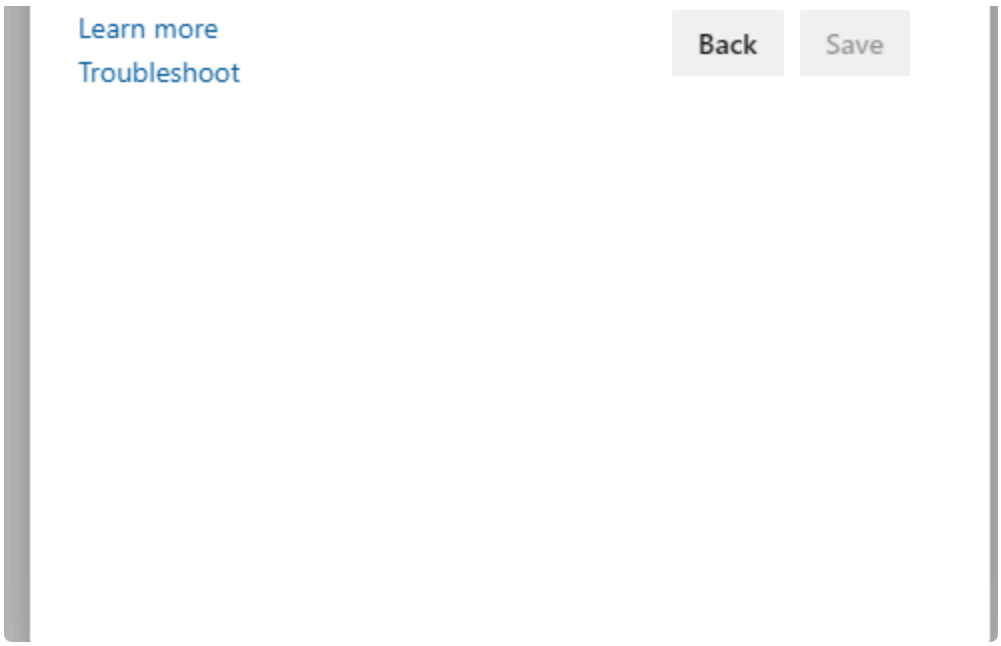
Step 4 – Service connection and RBAC

The service connection is the identity the pipeline uses to talk to Azure. It must be scoped to the whole subscription, and it must hold Contributor on that subscription.

Create the connection

1. Azure DevOps → project `king` → **Project settings** (gear icon, bottom-left corner).
2. Under **Pipelines** in the left list click **Service connections** → **Create service connection** (or **New service connection** if some exist).
3. Choose **Azure Resource Manager** → **Next**.
4. Identity type: **App registration (automatic)**. Credential: **Workload identity federation**. Click **Next**.
5. Scope level: **Subscription**.
6. Subscription: pick `Azure subscription 1 (fc6f4b31-…)`. A sign-in popup may appear; sign in with the same account.
7. Resource group: **leave empty**. If a value such as `myrg` is pre-filled, open the dropdown and clear it. A resource group here limits the identity to that group only, and the pipeline then cannot create `rg-fincorp-dev` or read state in `myrg`.
8. Service connection name: `sc-azure-fincorp` (this exact string is referenced in the YAML).
9. Tick **Grant access permission to all pipelines**.
10. Click **Save**. Azure DevOps now creates an app registration in Entra ID and a federated credential; this takes about 20 seconds.

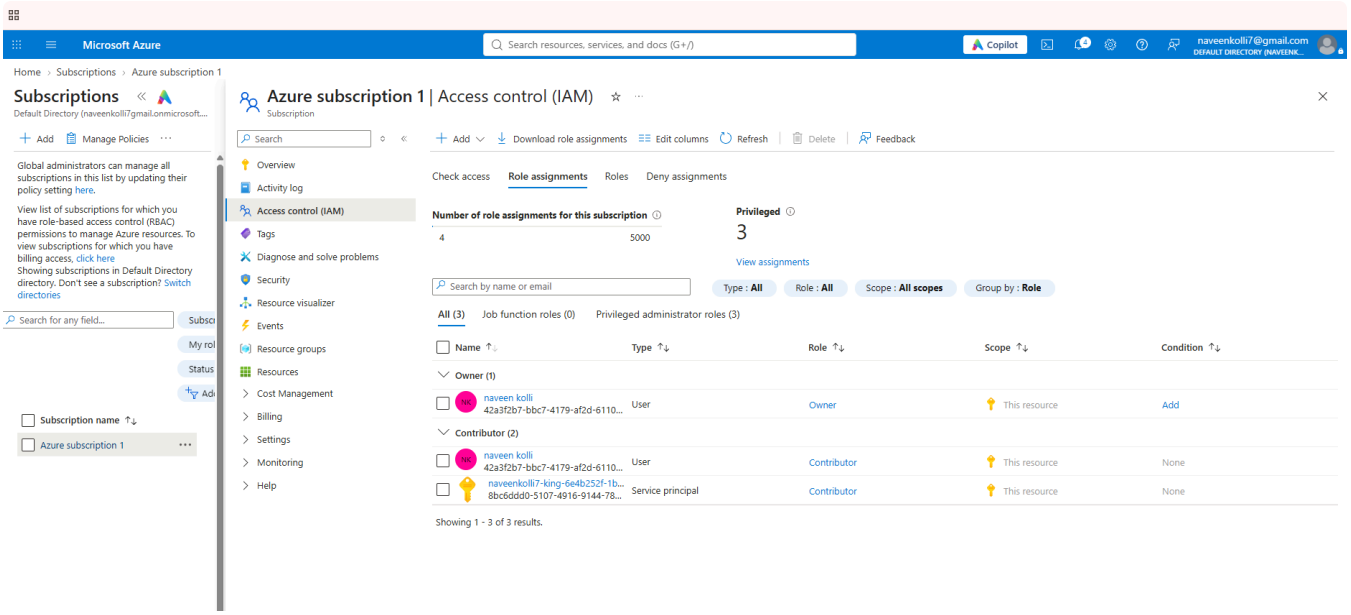

New Azure service connection
Azure Resource Manager
Identity type
App registration (automatic)
Recommended
Need help choosing a connection type?
Credential
Workload identity federation
Recommended
Scope level
Subscription
Management Group
Machine Learning Workspace
Subscription
Azure subscription 1 (fc6f4b31-69a1-46fd-a3dc-b6f8b776b00f)
If you can't find subscription you need, please create manually.
Resource group
myrg
Specify to limit access to the chosen resource group only.
Service Connection Name
Service Management Reference (optional)
Provide a Service Management Reference if your organization requires one to be configured on Entra ID App registrations.
Description (optional)
Security
Grant access permission to all pipelines



Verify the role in Azure

Saving the connection creates an app registration named `naveenkolli7-king-<guid>` and normally assigns it Contributor on the subscription. Confirm it before running anything:

1. Portal → search **Subscriptions** → click `Azure subscription 1`.
2. Left menu → **Access control (IAM)** → **Role assignments** tab.
3. In the search box type `king`. Expected row: Name `naveenkolli7-king-<guid>`, Type **Service principal**, Role **Contributor**, Scope **This resource**.
4. If the row is there, skip to Step 5.
5. If it is missing: **+ Add** → **Add role assignment**.
6. **Role** tab → click the **Privileged administrator roles** sub-tab → select **Contributor** → **Next**.
7. **Members** tab → Assign access to: **User, group, or service principal** → **+ Select members**.
8. In the right-hand search box type `naveenkolli7-king` → click the app that appears → **Select** → **Next**.
9. **Review + assign** → **Review + assign** again. A green notification confirms it.
10. Wait five minutes before running the pipeline; role assignments take time to propagate.



Step 5 – Pipeline YAML and first run

One YAML file runs the four Terraform commands on a Microsoft-hosted Ubuntu agent every time `main` changes.

1. Azure DevOps → **Pipelines** (rocket icon) → **Create Pipeline** (or **New pipeline**).
2. Where is your code? → **Azure Repos Git**.
3. Select a repository → `king`.
4. Configure your pipeline → **Starter pipeline**.
5. The editor opens with sample YAML. Select all (Ctrl+A), delete, and paste the YAML below.
6. Check the three state names in the `init` step match Step 1 (`myrg`, `stfincorptfstate`, `tfstate`) and that `sc:` matches the service connection name from Step 4.
7. Click **Save and run** (top right) → in the dialog keep "Commit directly to the main branch" → **Save and run**. This commits `azure-pipelines.yml` to the repo and starts run #1.
8. If the run shows "This pipeline needs permission to access a resource before this run can continue" → click **View** → **Permit** → **Permit**.
9. Click **Job** to watch the steps live. Each step turns green as it finishes.
10. Optional: rename the pipeline from "king" to `terraform-website` via ⋮ → **Rename/move**.

```
trigger:
- main

pool:
  vmImage: ubuntu-latest

variables:
  sc: sc-azure-fincorp

steps:
- task: TerraformInstaller@1
  inputs:
    terraformVersion: '1.9.5'

- task: TerraformTaskV4@4
  displayName: init
  inputs:
    provider: azurerm
    command: init
    backendServiceArm: $(sc)
    backendAzureRmResourceGroupName: myrg
    backendAzureRmStorageAccountName: stfincorptfstate
    backendAzureRmContainerName: tfstate
    backendAzureRmKey: webapp.tfstate

- task: TerraformTaskV4@4
  displayName: validate
  inputs:
    provider: azurerm
    command: validate

- task: TerraformTaskV4@4
  displayName: plan
  inputs:
    provider: azurerm
    command: plan
    environmentServiceNameAzureRM: $(sc)

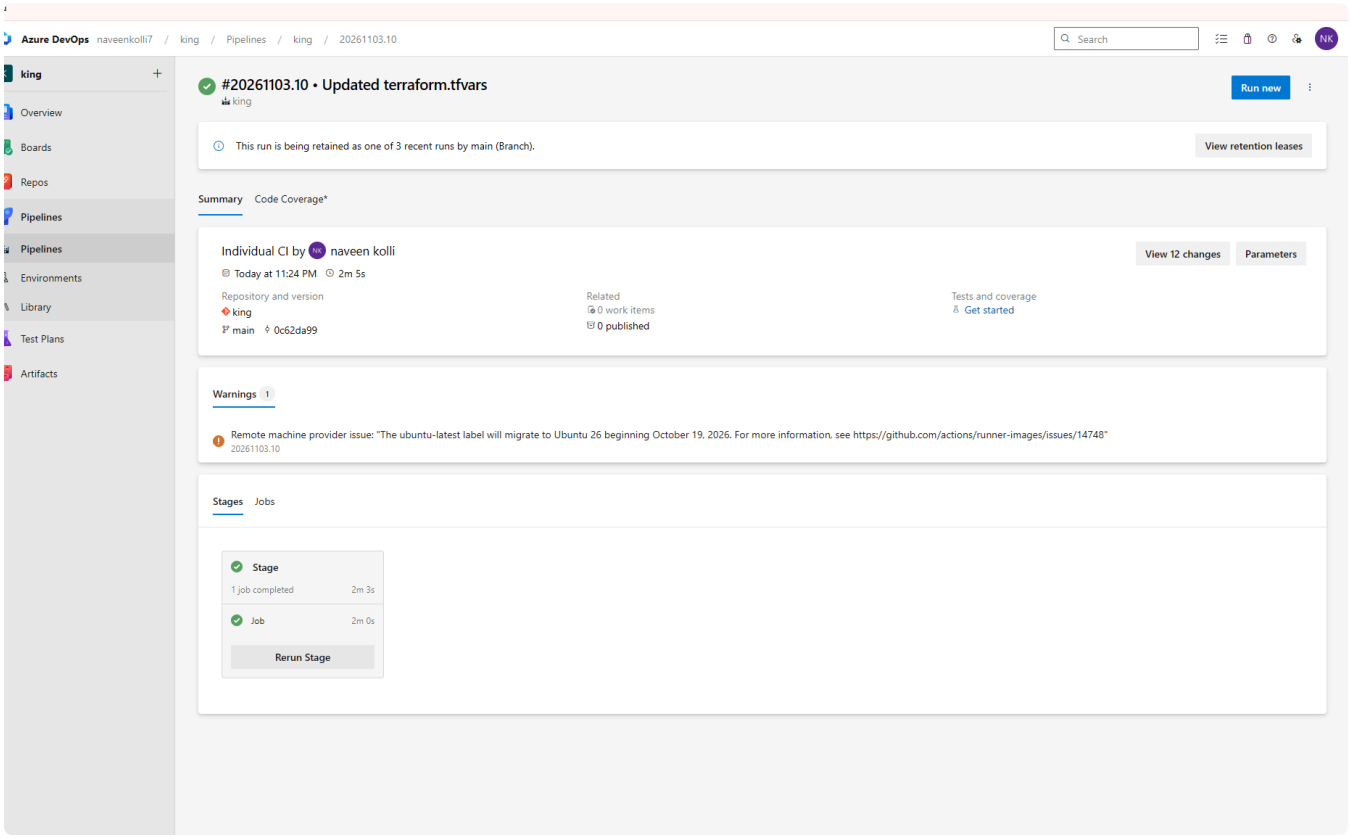
- task: TerraformTaskV4@4
  displayName: apply
  inputs:
```

```
    provider: azurerm
    command: apply
    environmentServiceNameAzureRM: $(sc)
    commandOptions: -auto-approve
```

What each step does:

Step	Command	What you see in the log
TerraformInstaller	downloads Terraform 1.9.5	version printed
init	connects to the state account, downloads the azurerm provider	“Successfully configured the backend”
validate	checks syntax of all `.tf` files	“Success! The configuration is valid”
plan	compares code with state and lists changes	`Plan: 4 to add, 0 to change, 0 to destroy`
apply	makes the changes	`Apply complete!` and `website_url = …`

A healthy run takes about two minutes and looks like this:



Step 6 – Verify the website and redeploy on change

The apply log ends with the public URL; open it and the page should load.

1. Pipelines → click the green run → **Job** → click the **apply** step on the left.
2. Scroll to the bottom of the log. The last lines are:

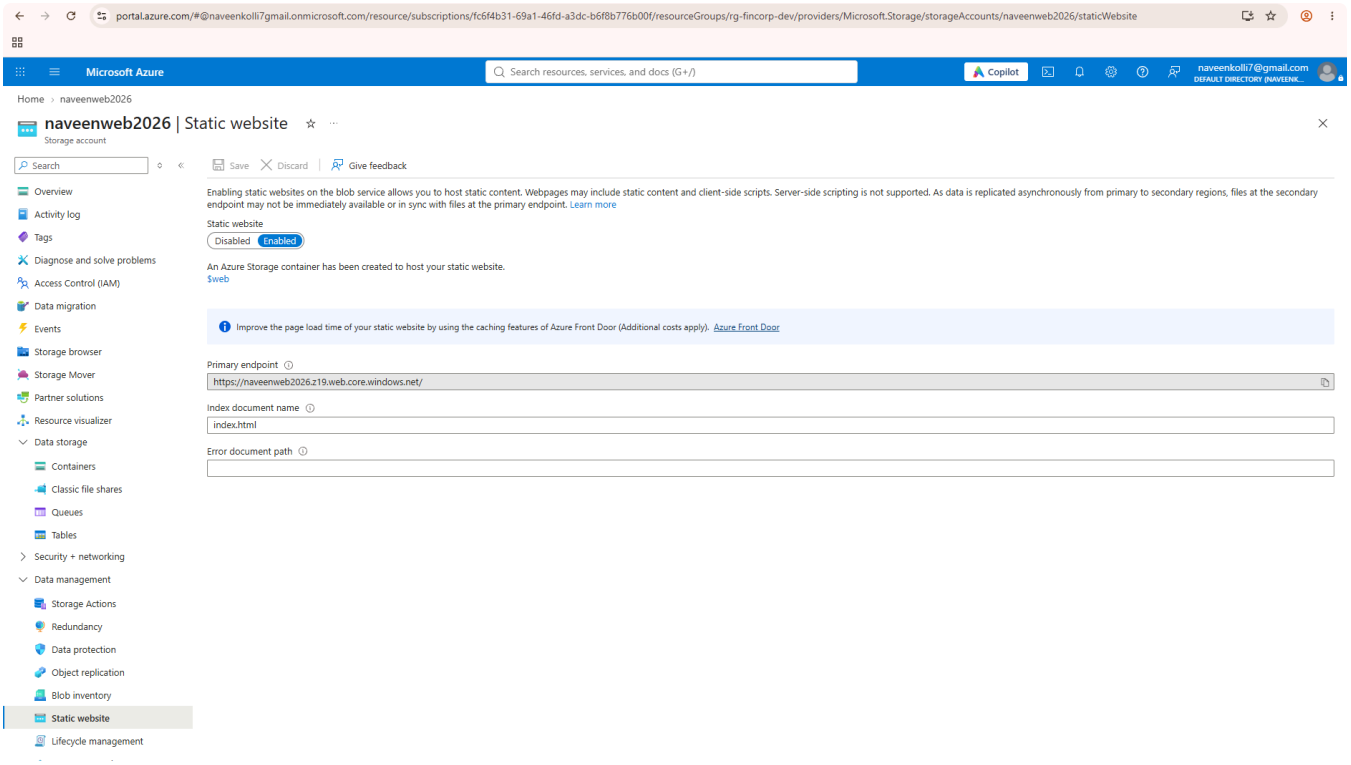
```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

website_url = "https://naveenweb2026.z19.web.core.windows.net/"
```

3. Copy the URL (without the quotes) into a browser. The FinCorp page loads.
4. Confirm in the portal: search **Storage accounts** → `naveenweb2026` → left menu **Data management → Static website**. Static website is **Enabled**, Index document name is `index.html`, and *Primary endpoint* is the same URL.
5. Confirm the file: same account → **Data storage → Containers** → `$web` → `index.html` is listed with Content type `text/html`.

6. Confirm the state: `stfincorptfstate` → Containers → `tfstate` → `webapp.tfstate` exists. This file is what lets the next run know these resources already exist.



The deployed page:

FinCorp

Accounts Services How this site runs

Banking that keeps its books in order.

FinCorp is a demonstration bank built to show how a regulated financial platform is provisioned, governed and operated on Azure — every resource defined in code, every change reviewed before it ships.

See the deployment path | Explore services

Account summary

Sample data · as of today

Everyday checking	$4,218.40
High-yield savings	$18,950.00
Interest this month	+$62.17
Pending card payments	−$340.00
Available	**$22,890.57**

How this page reached your browser

No one clicked through the Azure portal to build this. The page you are reading was shipped by a pipeline, from a Git commit to a public URL, in about two minutes.

Code committed

Terraform files pushed to Azure Repos — `webapp.tf`, `backend.tf`, variables.

Pipeline triggered

Azure Pipelines runs `init → validate → plan → apply` on a hosted Ubuntu agent.

Identity verified

Workload identity federation — no stored secrets. State is locked in Azure Storage.

Site published

Terraform creates the storage account, enables static hosting and uploads this page to `$web`.

What the platform is built to do

Storage administration	Blob containers, file shares and managed disks with lifecycle rules that move data from hot to cool to archive, soft delete and immutable retention for records that must not change.
Governance and access	Azure Policy enforces tagging and allowed regions; role-based access keeps every identity on least privilege; resource locks protect what must not be deleted.
Backup and recovery	Recovery Services vaults, geo-redundant storage and tested restores, so a regional outage is a drill the team has already run.
Cost and capacity	Budgets with alerts, orphaned-resource sweeps and right-sized tiers, reported monthly so spend never surprises anyone.

FinCorp is a fictional bank used for an Azure infrastructure portfolio project.

Built by Naveen Kolli · Terraform · Azure Pipelines · Azure Storage

Redeploying a change. Edit `index.html` in Repos (or locally and push). The commit triggers the pipeline, `plan` shows `1 to change` because `content_md5` differs, and `apply` re-uploads the blob. Refresh the URL; no other step is needed. Changes to any `.tf` file redeploy the same way.

Cleanup and next steps

The website account costs cents per month; the state account is shared by future projects and stays.

- To remove the website: in the YAML change the last step's `command: apply` to `command: destroy`, save, let it run once, then change it back. `rg-fincorp-dev` and `naveenweb2026` are deleted; `myrg` and `stfincorptfstate` remain.
- Next additions, in order of value for an infrastructure role: a separate **plan** stage and an approval-gated **apply** stage using an Environment; a second `.tfvars` for prod; Azure Policy and tagging in the same repo; a VM with managed disks once the subscription is upgraded.

Repository: dev.azure.com/naveenkolli7/king. Live site: naveenweb2026.z19.web.core.windows.net.